

Cmpe 492 - Presentation 2

Mustafa Ocak - Halil İbrahim Kasapoğlu
Advisor: Atay Ozgovde

Bogazici University



Role of networking in Kubernetes

- ▶ Kubernetes networking allows Kubernetes components to communicate with each other and with other applications.
- ▶ The Kubernetes platform is different from other networking platforms because it is based on a flat network structure that eliminates the need to map host ports to container ports.
- ▶ **Pods:** These are the smallest deployable units in Kubernetes, and they need to communicate with each other to ensure **the app works as intended**.
- ▶ **Nodes:** These are machines (virtual or physical) that run your applications, and they need to be connected for **workloads to scale** across multiple machines.
- ▶ **Cluster Services:** They expose applications to both **internal** and **external users**. Networking enables requests to reach the correct pod that can fulfill them.



Understanding Pod Networking

Pods in Kubernetes are assigned unique IP addresses within a cluster, enabling direct communication between them. By default, each pod is isolated and has its own IP address, which allows for secure communication and avoids port conflicts. These IP addresses are reachable only within the Kubernetes cluster network unless specific configurations are made for external access.



Main Communications

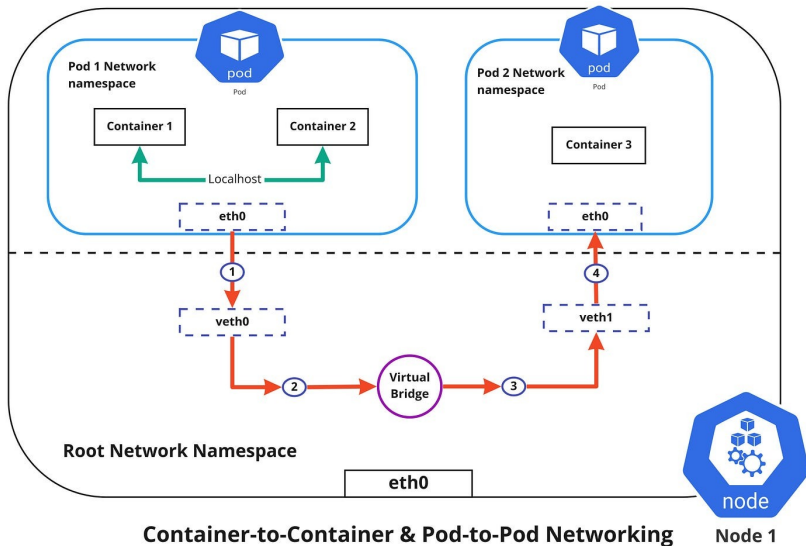
- ▶ Container-to-container communication
- ▶ Pod-to-pod communication
- ▶ Pod-to-service communication
- ▶ External-to-service communication

Kubernetes Networking Model

Pod-to-Pod Communication within the Same Node: Pods on the same node can communicate with each other using the node's network namespace. Since these pods are on the same physical or virtual machine, their communication can technically occur at Layer 2, with packets being forwarded based on MAC addresses within the node. This is typically managed by the container runtime and Kubernetes CNI (Container Network Interface) plugins, which set up the virtual Ethernet bridges or similar mechanisms that facilitate this L2 communication.



Diagram Pod-to-Pod



Container-to-Container & Pod-to-Pod Networking

Node 1

Figure 1:

Kubernetes Networking Model

Pod-to-Pod Communication across Nodes: When pods need to communicate across different nodes in the cluster, Kubernetes employs various networking solutions, such as Container Network Interfaces (CNIs) and software-defined networking (SDN) technologies. These solutions create a virtual network overlay that spans the entire cluster, enabling pod-to-pod communication across nodes. Some popular CNIs include Calico, Flannel, Weave, and Cilium. These networking solutions ensure that the pod's IP address remains reachable and provides transparent network connectivity regardless of the pod's location within the cluster.



Cluster-Internal Communication

By default, pods within a Kubernetes cluster can communicate with each other using their internal IP addresses. This communication happens over a virtual network overlay provided by the underlying container runtime or network plugin. The internal IP addresses are assigned by the Kubernetes cluster networking solution and are routable only within the cluster.

Kubernetes IP address ranges

Kubernetes clusters require allocating non-overlapping IP addresses for Pods, Services, and Nodes from a range of available addresses configured in the following components:

- ▶ The network plugin is configured to assign IP addresses to Pods.
- ▶ The kube-apiserver is configured to assign IP addresses to Services.
- ▶ The kubelet or the cloud-controller-manager is configured to assign IP addresses to Nodes.

Diagram

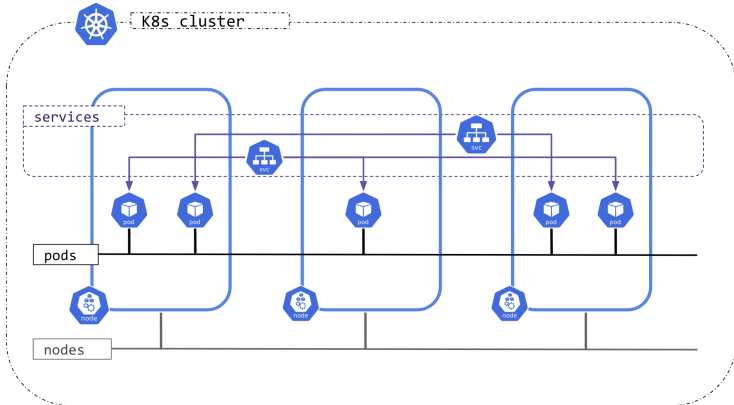


Figure 2: Kubernetes Cluster Network

Networking Types

Network plugin, kube-apiserver and kubelet/cloud-controller-manager are configured to assign Ip addresses with principals:

- ▶ IPv4 only
- ▶ IPv6 only
- ▶ Dual (All components must agree on the configured primary IP family.)

Kubernetes Services and Pod Networking

Kubernetes Services abstract pod networking by providing stable endpoints for communication between pods, ensuring reliable interaction regardless of the dynamic nature of pod lifecycles. Services act as a logical grouping of pods and provide a way to route traffic to the appropriate pod, even as individual pods are added or removed due to scaling or failure recovery.

- ▶ **ClusterIP:** This is the default service type. It creates an internal IP that can only be accessed within the Kubernetes cluster. ClusterIP ensures that pods can discover and communicate with each other internally without exposing any services to the external world. It's commonly used for internal microservice communication.

Example Diagram

ClusterIP

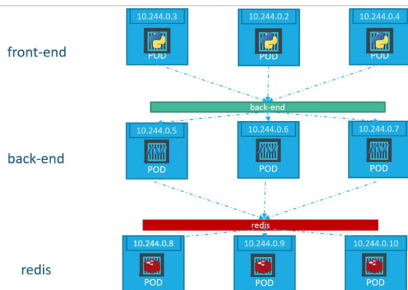


Figure 3: ClusterIp

Example YAML

```
apiVersion: v1
kind: Service
metadata:
  name: my-internal-service
spec:
  selector:
    app: my-app
  type: ClusterIP
  ports:
    - name: http
      port: 80
      targetPort: 80
      protocol: TCP
```

Figure 4: Enter Caption



Kubernetes Proxy

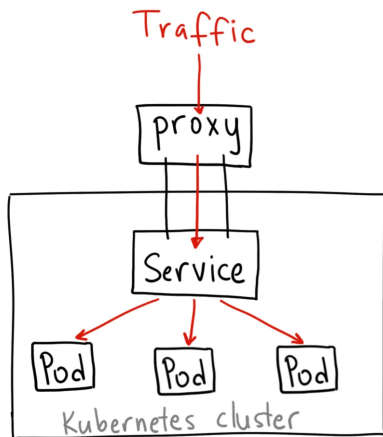


Figure 5: Kubernetes Proxy

Accessing Services via Kubernetes Proxy

Start the Kubernetes Proxy:

Command

```
$ kubectl proxy -port=8080
```

Now, you can navigate through the Kubernetes API to access the service using the following scheme:

Access Scheme

```
http://localhost:8080/api/v1/proxy/namespaces/<NAMESPACE>  
/services/<SERVICE-NAME>:<PORT-NAME>/
```

To access the service defined in the **default** namespace, for example:

Example URL

```
http://localhost:8080/api/v1/proxy/namespaces/default  
/services/my-internal-service:http/
```



When Would You Use the Kubernetes Proxy?

There are several scenarios where using the Kubernetes proxy to access your services is useful:

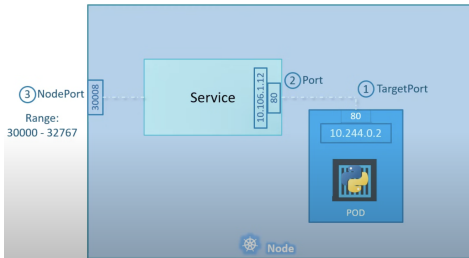
- ▶ **Debugging:** You can use the proxy to directly connect to your services from your laptop or local environment for debugging purposes. This is especially helpful when you need to test specific endpoints or behaviors in a controlled way.
- ▶ **Accessing Internal Services:** The proxy allows you to connect to internal services that are not exposed externally. For example, you might want to view internal dashboards or metrics that are only available within the cluster network.

Kubernetes Services and Pod Networking (Continued)

- ▶ **NodePort:** This service type exposes a service on the same port across each node in the cluster. The NodePort allocates a static port on each node (within a predefined range, usually 30000-32767). External traffic can reach the service by accessing any node's IP address combined with the NodePort. The kube-proxy ensures that the traffic is routed to the correct pod behind the service.

NodePort

Service - NodePort



```
service-definition.yml
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  type: NodePort
  ports:
    - targetPort: 80
      *port: 80
      nodePort: 30000
```

Figure 6: NodePort

Service - NodePort

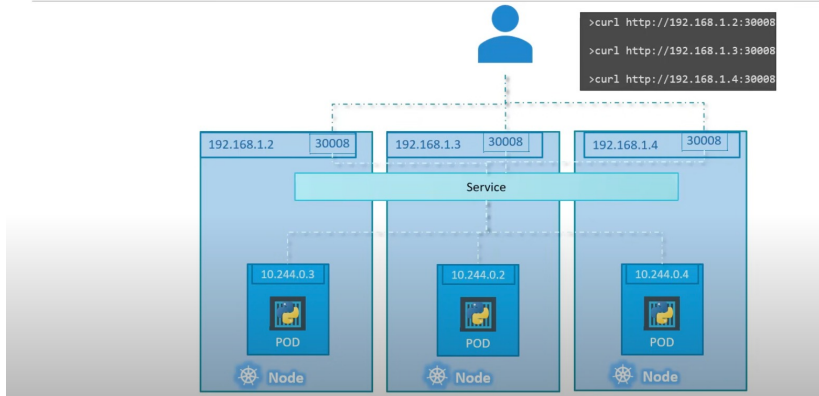


Figure 7: NodePort

NodePort vs ClusterIp

Basically, a NodePort service has two differences from a normal “ClusterIP” service. First, the type is “NodePort.” There is also an additional port called the nodePort that specifies which port to open on the nodes. If you don’t specify this port, it will pick a random port. Most of the time you should let Kubernetes choose the port

When Would You Use This?

There are many downsides to this method:

- ▶ You can only have one service per port
- ▶ You can only use ports 30000–32767
- ▶ If your Node/VM IP address change, you need to deal with that

For these reasons, It is generally not recommend using this method in production to directly expose the service.

Kubernetes Services and Pod Networking (Continued)

- ▶ **LoadBalancer:** This type creates an external load balancer that distributes incoming traffic across all the pods running the service. It is typically used in cloud environments (like AWS, GCP, or Azure), where a cloud provider's load balancer service can be provisioned. LoadBalancer services provide seamless scalability and allow external users to access the service through a single IP address that's managed by the cloud provider.

Diagram

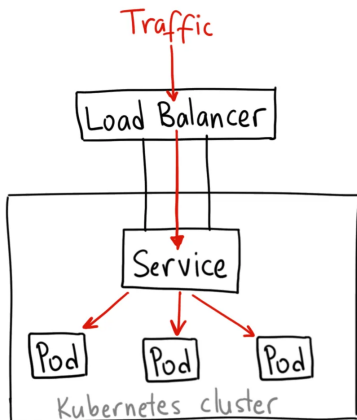


Figure 8: Load Balancer

How Does DNS (CoreDNS) Work in Kubernetes?

In a Kubernetes cluster, Services act as an abstraction layer, facilitating communication between different parts of an application.

- ▶ **Service DNS Mechanism:** Kubernetes Service DNS assigns DNS names to Services. This makes it easier for applications to dynamically discover and communicate with each other, without hardcoding IP addresses. **Typical DNS Name:**
`<service-name>.<namespace>.svc.cluster.local`
- ▶ **Automatic DNS Records:** When a Service is created, a DNS record is automatically generated and assigned to it. This DNS record is accessible within the cluster.
- ▶ **Resolving Service Names:** Other components within the cluster can resolve the Service's IP address by using its DNS name. This provides a seamless way for services to communicate with one another, even if the underlying IP addresses of the pods change.



How Does DNS Work in Kubernetes?

How DNS Resolution Works for Services:

- ▶ When a pod wants to communicate with another service, it performs a DNS lookup for the service name. This lookup is forwarded to the CoreDNS server running in the cluster.
- ▶ CoreDNS resolves the service name to its corresponding **ClusterIP**, which is the stable internal IP address assigned to the service. Kubernetes uses this IP to load balance traffic across all pods behind that service.
- ▶ DNS resolution also works across namespaces. For example, a pod in one namespace can communicate with a service in another namespace by using the fully qualified domain name (FQDN), such as
`<service-name>.<namespace>.svc.cluster.local`.
- ▶ CoreDNS handles the complexity of translating these service names into IP addresses, abstracting the dynamic and ephemeral nature of pods in the cluster.



What are Network Policies in Kubernetes?

Network Policies in Kubernetes:

- ▶ **Network Policies** are used in Kubernetes to control the flow of network traffic to and from pods. They allow cluster administrators and developers to define rules about which pods can communicate with each other and with external systems.
- ▶ By default, all pods in a Kubernetes cluster can communicate with each other, but network policies allow you to restrict this communication based on specified rules.

Network Policies in Kubernetes (Contd)

How Network Policies Are Defined and Enforced:

- ▶ Network policies are defined using YAML manifests, just like other Kubernetes resources. These policies specify which traffic is allowed (or denied) based on pod labels, namespaces, and other criteria.
- ▶ Once a network policy is applied to a pod, the Kubernetes **Container Network Interface (CNI)** plugin (e.g., Calico, Cilium, Weave) is responsible for enforcing the rules by configuring the network layer (e.g., through iptables).
- ▶ Network policies define two primary types of traffic rules:
 - ▶ **Ingress Rules:** Control the incoming traffic to a pod.
 - ▶ **Egress Rules:** Control the outgoing traffic from a pod.

Example: Ingress Rule

Ingress Rule Example:

- ▶ This policy allows only traffic from pods labeled 'app=frontend' to reach pods labeled 'app=backend'.

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
      - podSelector:
          matchLabels:
            app: frontend
```

Figure 9:

Example: Egress Rule

Egress Rule Example:

- ▶ This policy restricts the 'backend' pods from sending traffic to any external IP except for the allowed IP range (e.g., 10.0.0.0/24).

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: restrict-egress
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Egress
  egress:
    - to:
        - ipBlock:
            cidr: 10.0.0.0/24
```

Figure 10: Enter Caption

What are CNI Plugins in Kubernetes?

CNI (Container Network Interface) plugins are responsible for managing networking for pods in Kubernetes. They define how network interfaces are configured for containers, ensuring communication between pods within the same node and across different nodes in the cluster.

- ▶ CNI plugins handle tasks such as IP address assignment, routing, and network isolation.
- ▶ Kubernetes does not provide a built-in networking solution, so it relies on various CNI plugins to implement networking policies and ensure network connectivity.

Different CNI Plugins and Their Impact on Networking

Several CNI plugins are commonly used in Kubernetes, each offering different features and performance characteristics:

- ▶ **Flannel:** Simple and lightweight; provides basic networking using an overlay network or direct routing. It is often chosen for smaller clusters or environments where complex network policies are not required.
- ▶ **Calico:** Offers advanced networking features, including support for network policies, BGP routing, and native IP networking (without overlays). Calico is widely used in production environments that need robust networking and security features.

More CNI Plugins

- ▶ **Weave:** Focuses on simplicity and supports fast pod-to-pod networking. It provides encryption for traffic between pods and can be set up quickly for basic networking needs.
- ▶ **Cilium:** Built with security in mind, Cilium uses eBPF (extended Berkeley Packet Filter) for efficient networking and security at the kernel level. It provides deep visibility into network traffic and is suitable for microservice-based architectures.

Impact of CNI Plugins on Pod Networking

The choice of CNI plugin influences several aspects of pod networking:

- ▶ **Performance:** Plugins like Calico (with native IP routing) tend to offer better performance than overlay-based solutions like Flannel, especially in larger clusters.
- ▶ **Network Policies:** Some plugins, such as Calico and Cilium, offer robust support for Kubernetes Network Policies, providing advanced security features and traffic control.
- ▶ **Scalability:** For large-scale clusters, choosing a plugin that supports efficient routing and scalability (e.g., Calico or Cilium) is crucial to maintaining network performance.

How Do Ingress Resources Work in Kubernetes?

Ingress resources in Kubernetes are used to manage external access to services within the cluster. They provide rules to route external HTTP/HTTPS traffic to specific services based on URL paths or hostnames.

- ▶ Ingress allows you to define how traffic should be routed from outside the cluster to internal services, typically used for web-based services.
- ▶ Unlike other service types (e.g., NodePort or LoadBalancer), which expose services on specific ports or IP addresses, Ingress provides a centralized way to manage and route traffic across multiple services.

Diagram

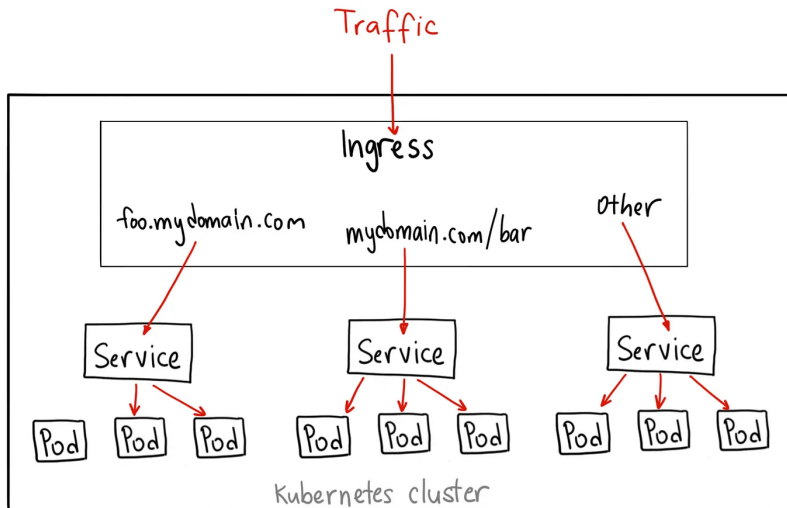


Figure 11: Ingress

Example Yaml

```
apiVersion: extensions/v1beta1
kind: Ingress
metadata:
  name: my-ingress
spec:
  backend:
    serviceName: other
    servicePort: 8080
  rules:
    - host: foo.mydomain.com
      http:
        paths:
          - backend:
              serviceName: foo
              servicePort: 8080
    - host: mydomain.com
      http:
        paths:
          - path: /bar/*
            backend:
              serviceName: bar
              servicePort: 8080
```

Figure 12: Yaml



Ingress Controllers

Ingress Controllers are the key components that make Ingress resources work. While the Ingress resource defines the rules for traffic routing, the Ingress controller is responsible for enforcing those rules.

- ▶ An Ingress controller is a specialized application or pod running in the Kubernetes cluster that watches for Ingress resources and configures the underlying load balancer or proxy accordingly.
- ▶ Popular Ingress controllers include **NGINX Ingress**, **HAProxy Ingress**, and cloud provider-specific controllers like **GCE Ingress** for Google Cloud.



How Ingress Routes External Traffic

Ingress rules define how traffic from outside the cluster should be routed to internal services. Ingress controllers listen to these rules and act accordingly.

- ▶ Ingress routes traffic based on several criteria, including:
 - ▶ **Hostnames:** Traffic can be routed based on specific domain names (e.g., `myapp.example.com`).
 - ▶ **Paths:** Traffic can also be routed based on URL paths (e.g., `/api`, `/dashboard`).
- ▶ The Ingress controller handles the routing of this traffic to the appropriate service, load balancing the requests to the pods behind that service.

What is kube-proxy?

kube-proxy is a key networking component in Kubernetes that runs on each node in the cluster. Its primary role is to manage and route traffic between services and pods within the cluster.

kube-proxy ensures that network traffic destined for a service reaches the appropriate pods, regardless of where they are running.

- ▶ kube-proxy runs on each Kubernetes node and maintains network rules.
- ▶ It ensures communication between services and the pods that back them.
- ▶ kube-proxy can use either **iptables** or **IPVS** to manage network traffic, depending on the size and performance requirements of the cluster.

How kube-proxy Works

kube-proxy works by monitoring Kubernetes services and creating the necessary network rules to forward traffic to the appropriate pods.

- ▶ When a service is created, kube-proxy creates rules to direct traffic to the service's backend pods.
- ▶ kube-proxy can handle different types of service traffic: TCP, UDP, and SCTP.
- ▶ kube-proxy maintains connectivity and load balances traffic between multiple pods to ensure high availability.

Modes of Operation:

- ▶ **iptables**: Works by setting up network rules using Linux's built-in iptables utility.
- ▶ **IPVS**: A more scalable, high-performance option that uses the Linux kernel's IPVS technology.



kube-proxy with iptables

iptables mode:

- ▶ **iptables** is a Linux utility that manages packet filtering rules.
- ▶ kube-proxy in iptables mode creates and manages a set of iptables rules for each service in the cluster.
- ▶ Each service gets an entry in the iptables rule set, mapping the service's **ClusterIP** to the corresponding pod's IP address.
- ▶ **How it works:** When a request is sent to a service's ClusterIP, the iptables rules direct the traffic to one of the available pod IPs backing the service.

kube-proxy with IPVS

IPVS mode:

- ▶ **IPVS (IP Virtual Server)** is a more efficient and scalable alternative to iptables, especially in large Kubernetes clusters.
- ▶ kube-proxy in IPVS mode uses the Linux kernel's IPVS technology to create virtual servers for each Kubernetes service.
- ▶ **How it works:** IPVS creates a virtual server for each service, which distributes traffic to real servers (pods) using various load-balancing algorithms (e.g., round-robin, least connections, etc.).
- ▶ IPVS is more efficient for managing larger clusters and provides faster performance with lower overhead than iptables.

Comparison: iptables vs IPVS

Comparison between iptables and IPVS:

▶ iptables:

- ▶ Simpler and widely supported across Linux distributions.
- ▶ Works well for small to medium-sized clusters.
- ▶ Less efficient at handling large numbers of services and pods.

▶ IPVS:

- ▶ Ideal for larger clusters with many services and pods.
- ▶ More scalable and efficient, especially for high-traffic environments.
- ▶ Supports multiple load-balancing algorithms (e.g., round-robin, least connections).

▶ Which one to use?

- ▶ By default, kube-proxy uses iptables, but you can configure it to use IPVS for improved scalability and performance in larger clusters.